Cyprus University of Technology

CEI 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Εισαγωγή στην Java

Παναγιώτης Ηλία

Εξέλιξη των γλωσσών προγραμματισμού

- Η εξέλιξη των γλωσσών προγραμματισμού είναι μια διαδικασία αφαίρεσης
  - Στην αρχή ένα πρόγραμμα ήταν μια σειρά από εντολές σε γλώσσα μηχανής
  - Με τον Διαδικαστικό Προγραμματισμό (procedural programming), ένα πρόγραμμα έγινε μια συλλογή από διαδικασίες που η μία καλεί την άλλη
  - Στον Συναρτησιακό Προγραμματισμό (functional programming) ένα πρόγραμμα είναι μια συλλογή από συναρτήσεις όπου η μία εφαρμόζεται πάνω στην άλλη
  - Στον Λογικό Προγραμματισμό (logic programming) ένα πρόγραμμα είναι μια συλλογή από κανόνες και γεγονότα
  - Στον Αντικειμενοστρεφή Προγραμματισμό (object-oriented programming) ένα πρόγραμμα είναι μια συλλογή από κλάσεις και αντικείμενα όπου το ένα μιλάει με το άλλο

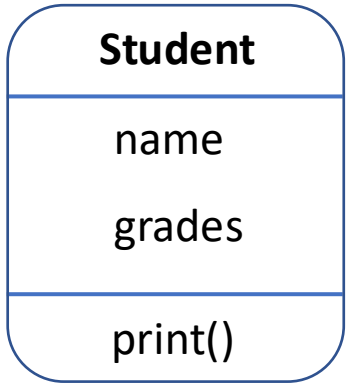
Αντικειμενοστρεφής Προγραμματισμός

- Οι πέντε αρχές του Alan Kay:
  - Τα πάντα είναι αντικείμενα
  - Ένα πρόγραμμα είναι μια συλλογή από αντικείμενα όπου το ένα λέει στο άλλο τι να κάνει
  - Κάθε αντικείμενο έχει δικιά του μνήμη και αποτελείται από άλλα αντικείμενα
  - Κάθε αντικείμενο έχει ένα συγκεκριμένο τύπο
    - Τύπος = Κλάση
  - Αντικείμενα του ίδιου τύπου μπορούν να δεχτούν τα ίδια μηνύματα
    - Δηλαδή έχουν τις ίδιες λειτουργίες

Κλάσεις και Αντικείμενα

Κλάση

Μια αφηρημένη περιγραφή μιας οντότητας



Όνομα κλάσης

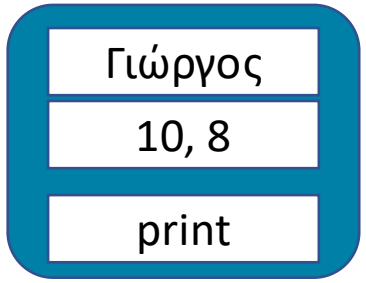
Πεδία κλάσης: Ιδιότητες/Χαρακτηριστικά

Μέθοδοι κλάσης: λειτουργίες

Αντικείμενα

Ένα συγκεκριμένο στιγμιότυπο της κλάσης

Student studentGeorge:



Πρόσβαση στο αντικείμενο μόνο μέσω **κλήσεων των μεθόδων**:

studentGeorge.print()

Τυπώνει τις πληροφορίες για το αντικείμενο.

Σύντομη ιστορία του Αντικειμενοστρεφούς Προγραμματισμού

- Η πρώτη γλώσσα που χρησιμοποίησε τις έννοιες της κλάσης και του αντικειμένου θεωρείται η SIMULA (1960s)
  - Γλώσσα για προσομοιώσεις συστημάτων
- Εμπνευσμένος από την SIMULA ο Alan Kay δημιούργησε στην HP την γλώσσα SmallTalk με στόχο μια γλώσσα που να υποστηρίζει γραφικά (1970s)
  - Ήταν αυτός που εισήγαγε την έννοια «Αντικειμενοστραφής Προγραμματισμός» (Object Oriented Programming)
  - Το 2003 βραβεύτηκε με το Turing Award
- Οι ιδέες του αντικειμενοστραφούς προγραμματισμού άρχισαν να εισάγονται σε πολλές υπάρχουσες η νέες γλώσσες. Ο Bjorn Stroustrup δημιούργησε την C++ (1980s)
- Η Sun δημιούργησε την γλώσσα Java (1990s)

Η γλώσσα προγραμματισμού JAVA

- Γλώσσα Προγραμματισμού 3ης Γενιάς
  - Δημιουργήθηκε το 1991 απο την “Green Team” της Sun Microsystems. Επίσημη ανακοίνωση της Γ.Π. το 1995
  - Ο στόχος ήταν ο εύκολος προγραμματισμός ηλεκτρονικών συσκευών και συστημάτων για ευρεία κατανάλωση
- Βασισμένη στη C και C++ αλλά με σημαντικές αλλαγές και “βελτιώσεις”
- Επιτρέπει στους χρήστες να αναπτύσσουν εφαρμογές για διαδίκτυο, εξυπηρετητές, υπολογιστές και κινητές συσκευές

  ➔ είναι γλώσσα προγραμματισμού γενικού σκοπού

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- **Απλή**
- Αντικειμενοστρεφής
- Κατανεμημένη
- Interpreted
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- Πολύ-νηματική
- Δυναμική

Η JAVA έχει μοντελοποιηθεί σύμφωνα με την C και C++ αλλά αντιθέτως με την C++ η JAVA δεν έχει header files, pointer arithmetic, pointer syntax, structures, unions, operator overloading, virtual base classes, κ.α.)

Μερικοί αναφέρονται στην Java σαν "C++--" γιατί είναι παρόμοια με την C++ αλλά με περισσότερη λειτουργικότητα και λιγότερα αρνητικά σημεία.

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- **Αντικειμενοστρεφής**
- Κατανεμημένη
- Interpreted
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- Πολύ-νηματική
- Δυναμική

Η JAVA έχει σχεδιαστεί από την αρχή για να είναι αντικειμενοστρεφή αντίθετα με άλλες γλώσσες.

Ο Α/Σ προγραμματισμός (Object-Oriented Programming (OOP)) είναι δημοφιλής και συνεχώς αντικαθιστά τις παραδοσιακές διαδικαστικές τεχνικές.

Ένα κύριο θέμα του OOP είναι το πώς να επαναχρησιμοποιείται ο κώδικας. Παρέχει μεγαλύτερη ευελιξία, σπονδυλωτή οργάνωση μέσω τεχνικών όπως πολυμορφισμός και κληρονομικότητα.

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- **Κατανεμημένη**
- Interpreted
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- Πολύ-νηματική
- Δυναμική

Η JAVA έχει σχεδιαστεί ώστε να υποστηρίζει εύκολα την επικοινωνία μεταξύ δικτυωμένων προγραμμάτων/υπολογιστών.

Δεδομένου ότι η λειτουργικότητα επικοινωνίας είναι εμπεδωμένη στην JAVA, η επικοινωνία μεταξύ δικτυωμένων προγραμμάτων είναι παρόμοια με προγράμματα που γράφουν και διαβάζουν από αρχεία.

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- Κατανεμημένη
- **Interpreted**
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- Πολύ-νηματική
- Δυναμική

Χρειάζεται διερμηνέας για να τρέξει ένα JAVA πρόγραμμα. Τα προγράμματα μεταγλωττίζονται πρώτα σε κώδικα τύπου Java Virtual Machine (bytecode). Το bytecode είναι ανεξάρτητο μηχανής και μπορεί να τρέξει σε οποιαδήποτε μηχανή που έχει διερμηνέα Java (interpreter).

Σημειώστε ότι ο διερμηνέας είναι μέρος της Java Virtual Machine (JVM).

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- Κατανεμημένη
- Interpreted
- **Εύρωστη**
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- Πολύ-νηματική
- Δυναμική

Οι JAVA μεταγλωττιστές αναγνωρίζουν διάφορα προβλήματα που άλλες γλώσσες τα αναγνωρίζουν μετά την εκτέλεση.

Έχει καλύτερο έλεγχο τύπων και προγραμματιστικών δομών.

Έχει έλεγχο εξαιρέσεων για να εξασφαλίσει μεγαλύτερο βαθμό ευρωστίας.

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- Κατανεμημένη
- Interpreted
- Εύρωστη
- **Ουδέτερη Αρχιτεκτονικής**
- Φορητή
- Πολύ-νηματική
- Δυναμική

Ένα γνωστό μότο της JAVA είναι

"Write once, run anywhere"

Ένα JAVA πρόγραμμα μπορεί να τρέξει οπουδήποτε υπάρχει Java Virtual Machine (JVM)!

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- Κατανεμημένη
- Interpreted
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- **Φορητή**
- Πολύ-νηματική
- Δυναμική

Ένα JAVA πρόγραμμα δεν χρειάζεται να μεταγλωττιστεί ξανά για να τρέξει!

Τα χαρακτηριστικά της JAVA

Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- Κατανεμημένη
- Interpreted
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- **Πολύ-νηματική**
- Δυναμική

Ο πολύ-νηματικός προγραμματισμός (multithread programming) είναι εμπεδωμένος μέσα στους κύριους μηχανισμούς της JAVA.

Δεν χρειάζονται ρυθμίσεις και ειδικά καλέσματα στο λειτουργικό για να τρέξουν πολύ-νηματικά προγράμματα.

Τα χαρακτηριστικά της JAVA

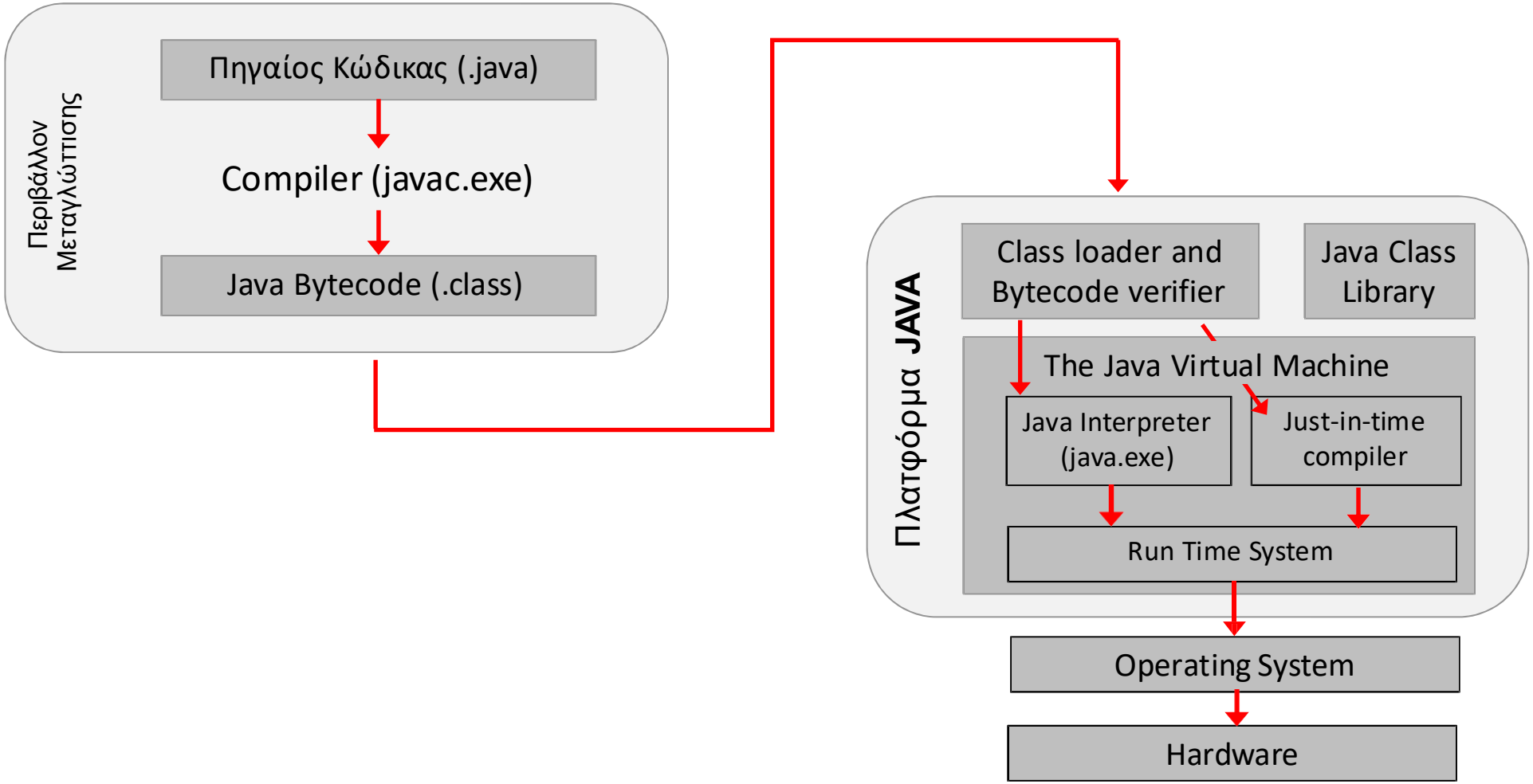
Η JAVA είναι:

- Απλή
- Αντικειμενοστρεφής
- Κατανεμημένη
- Interpreted
- Εύρωστη
- Ουδέτερη Αρχιτεκτονικής
- Φορητή
- Πολύ-νηματική
- **Δυναμική**

Η JAVA έχει σχεδιαστεί ώστε να προσαρμόζεται σε ένα δυναμικό και εξελισσόμενο περιβάλλον.

Καινούριος κώδικας μπορεί να φορτωθεί χωρίς να χρειάζεται επανα-μεταγλώττιση. Ως αποτέλεσμα οι χρήστες δεν χρειάζονται να εγκαταστήσουν καινούριες εκδόσεις λογισμικού κάθε φορά που υπάρχει αλλαγή.

Η πλατφόρμα της JAVA



Η πλατφόρμα της JAVA

- Το μεγαλύτερο πλεονέκτημα της Java είναι η μεταφερσιμότητα (portability). Ο κώδικας μπορεί να τρέξει πάνω σε οποιαδήποτε πλατφόρμα
  - **Write-Once-Run-Anywhere** μοντέλο, σε αντίθεση με το σύνηθες Write-Once-Compile-Anywhere μοντέλο
- Αυτό επιτυγχάνεται δημιουργώντας ένα ενδιάμεσο κώδικα (bytecode) ο οποίος μετά τρέχει πάνω σε μια εικονική μηχανή (Java Virtual Machine) η οποία το μεταφράζει σε γλώσσα μηχανής
  - Οι προγραμματιστές πλέον γράφουν κώδικα για την εικονική μηχανή, η οποία δημιουργείται για οποιαδήποτε πλατφόρμα

Πλεονεκτήματα και Μειονεκτήματα της JAVA

- Ξεκινάς εύκολα: Α/Σ προγραμματισμός με πολλές ομοιότητες με τη C και τη C++ στο συντακτικό
- Γράφεις λιγότερο κώδικα: μετρήσεις δείχνουν ότι τα προγράμματα JAVA μπορεί να είναι 4 φορές μικρότερα από αντίστοιχα προγράμματα C++
- Γράφεις καλύτερο κώδικα: ενθαρρύνει τις καλές προγραμματιστικές πρακτικές, αυτόματη διαχείριση μνήμης (αποκομιστής σκυβάλων)
- Γρήγορη κατασκευή κώδικα: απλούστερη γλώσσα από την C++
- Ανεξαρτησία αρχιτεκτονικής - γράφεις κώδικα μια φορά, τρέχει παντού (write once, run anywhere)
- Ευκολότερη εγκατάσταση και διανομή κώδικα

Πλεονεκτήματα και **Μειονεκτήματα** της JAVA

- **Επίδοση**
  - Λόγω της χρήσης του διερμηνέα και της εικονικής μηχανής, η εκτέλεση προγραμμάτων στην πλατφόρμα της JAVA μπορεί είναι πιο αργή από την εκτέλεση ιθαγενή κώδικα (native code).
  - Ωστόσο, η πρόοδος σε τεχνολογίες μεταγλωττιστών και εικονικών μηχανών, έχει βελτιώσει την επίδοση της JAVA χωρίς να βλάπτει την φορητότητα της γλώσσας.
- **Γλώσσα υψηλότερου επιπέδου από τη C/C++**
  - Ο προγραμματιστής έχει πιο περιορισμένες δυνατότητες διαχείρισης μνήμης και σχετικών βελτιστοποιήσεων.
  - Ο προγραμματιστής έχει περιορισμένες δυνατότητες άμεσης πρόσβασης στο υλικό του Η/Υ.

HELLO WORLD

Το πρώτο μας πρόγραμμα σε Java

Δομή ενός απλού Java προγράμματος

- Το όνομα του αρχείου που κρατάει το πρόγραμμα είναι X.java (όπου X το όνομα του προγράμματος)
  - Στο παράδειγμα μας ονομάζουμε το πρόγραμμα μας: HelloWorld.java
- Μέσα στο πρόγραμμα μας πρέπει να έχουμε μια κλάση με το όνομα X
  - `class X` (`class HelloWorld` στο παράδειγμα μας)
- Η κλάση X θα πρέπει να περιέχει μια μέθοδο main η οποία είναι το σημείο εκκίνησης του προγράμματος μας
  - `public static void main(String[] args)`

Αρχείο HelloWorld.java

```
class HelloWorld
{
    public static void main(String args[])
    {
        // print message
        System.out.println("Hello world!");
    }
}
```

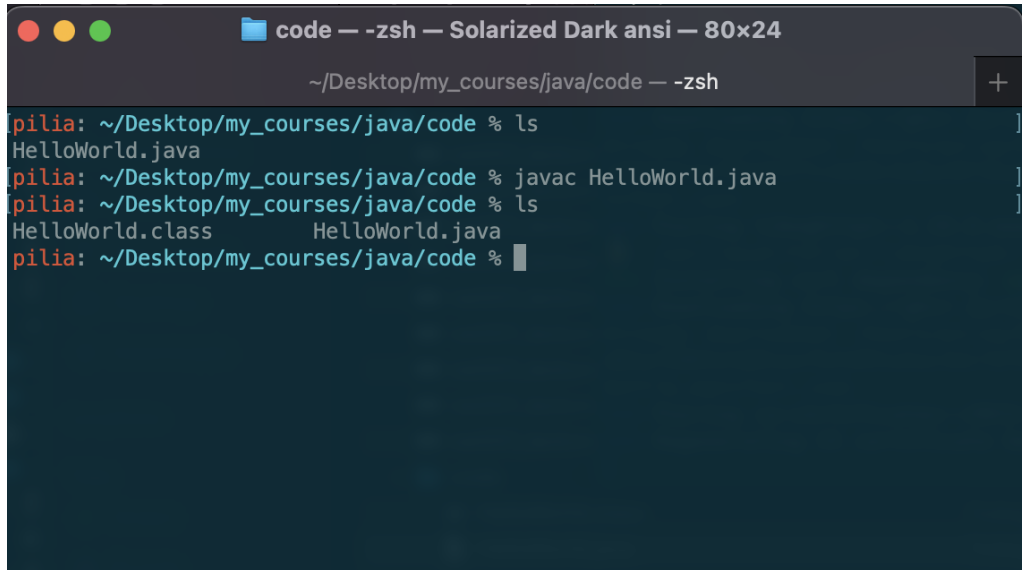
Το όνομα του .java αρχείου και το όνομα της κλάσης (που περιέχει την μέθοδο main) θα πρέπει να είναι πάντα τα ίδια!

Μεταγλώττιση – Compiling

- Η μεταγλώττιση γίνεται με την εντολή javac
  - javac < .java αρχείο>

Π.χ.

```
➢ javac HelloWorld.java
```



- Το αποτέλεσμα είναι η δημιουργία ενός .class αρχείου που περιέχει τον ενδιάμεσο κώδικα (bytecode)
- Το αρχείο HelloWorld.class στο παράδειγμα μας

Εκτέλεση - Running

- Η εκτέλεση του κώδικα γίνεται με την εντολή java
  - java <όνομα αρχείου χωρίς επίθεμα>

```
➢ java HelloWorld
```

Χωρίς κανένα επίθεμα!



```
class HelloWorld
{
    public static void main(String args[])
    {
        // print message
        System.out.println("Hello world!");
    }
}
```

Λέξεις σε κόκκινο: δεσμευμένες λέξεις

Ορίζει την κλάση

Όνομα της κλάσης

```
class HelloWorld
{
      public static void main(String args[])
      {
            // print message
            System.out.println("Hello world!");
      }
}
```

```
class HelloWorld
{
    public static void main(String args[])
    {
        // print message
        System.out.println("Hello world!");
    }
}
```

- Τα άγκιστρα { … } ορίζουν ένα λογικό block του κώδικα
- Αυτό μπορεί να είναι μία κλάση, μία συνάρτηση, ένα if statement
- Οι μεταβλητές που ορίζουμε μέσα σε ένα λογικό block, έχουν εμβέλεια μέσα στο block
- Αντίστοιχο των tabs στην Python, εδώ δεν χρειάζονται αλλά είναι καλό να τα βάζουμε για να διαβάζεται ο κώδικας πιο εύκολα

Ορισμός της συνάρτησης main

```
class HelloWorld
{
        public static void main(String args[])
        {
                // print message
                System.out.println("Hello world!");
        }
}
```

Ορισμός της συνάρτησης main

```
class HelloWorld
{
        public static void main(String args[])
        {
                // print message
                System.out.println("Hello world!");
        }
}
```

public, static: Θα τα εξηγήσουμε αργότερα

Ορισμός της συνάρτησης main

```
class HelloWorld
{
      public static void main(String args[])
      {
            // print message
            System.out.println("Hello world!");
      }
}
```

Το τι επιστρέφει η μέθοδος

void: Η μέθοδος δεν επιστρέφει τίποτα

Ορισμός της συνάρτησης main

```
class HelloWorld
{
        public static void main(String args[])
        {
                // print message
                System.out.println("Hello world!");
        }
}
```

Το όνομα της μεθόδου

- main: Ειδική περίπτωση που σηματοδοτεί το σημείο εκκίνησης του προγράμματος.

Ορισμός της συνάρτησης main

```
class HelloWorld
{
      public static void main(String args[])
      {
            // print message
            System.out.println("Hello world!");
      }
}
```

Ορίσματα της μεθόδου

- Ένας πίνακας από Strings που αντιστοιχούν στις παραμέτρους με τις οποίες τρέχουμε το πρόγραμμα.

```
class HelloWorld
{
        public static void main(String args[])
        {
                // print message
                System.out.println("Hello world!");
        }
}
```

Η κλάση String

- String: κλάση που χειρίζεται τα αλφαριθμητικά
- Στη Java χρειάζεται να ορίσουμε τον τύπο της κάθε μεταβλητής
- Strongly typed language

Σχόλια!

```
/**
 * A class that prints a message "hello world"
 **/
class HelloWorld
{
      public static void main(String args[])
      {
            // print message
            System.out.println("Hello world!");
      }
}
```

```
class HelloWorld
{
    public static void main(String args[])
    {
            // print message
            System.out.println("Hello world!");
     }
}
```

Κάθε εντολή στη Java πρέπει να τερματίζει με το ;

```
class HelloWorld
{
    public static void main(String args[])
    {
        // print message
        System.out.println("Hello world!");
    }
}
```

Αντικείμενο `System.out`
Ορίζει το ρεύμα εξόδου

Μέθοδος `println`:
Τυπώνει το String αντικείμενο που δίνεται ως όρισμα και αλλάζει γραμμή

```
class HelloWorld
{
    public static void main(String args[])
    {
        // print message
        System.out.println("Hello world!");
    }
}
```

Το “Hello World” είναι ένα αντικείμενο της κλάσης String

Programming Style

Το όνομα της κλάσης ξεκινάει με κεφαλαίο και χρησιμοποιούμε την CamelCase σύμβαση

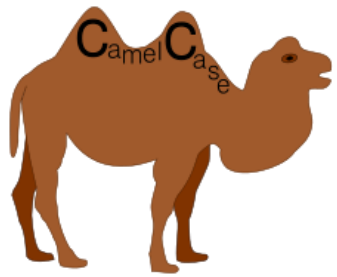
```
class HelloWorld
{
    public static void main(String args[])
    {
        // print message
        System.out.println("Hello world!");
    }
}
```

Στοίχιση του κώδικα:

- Οι εντολές μέσα σε ένα block του κώδικα ξεκινάνε ένα tab πιο μπροστά από το προηγούμενο.
- Όλες οι εντολές σε ένα block είναι στοιχισμένες
- Τα άγκιστρα είναι στοιχισμένα με την εντολή που ορίζει το block

Programming Style: Ονόματα

- Τα ονόματα των κλάσεων ξεκινάνε με κεφαλαίο, τα ονόματα των πεδίων, μεθόδων και αντικειμένων με μικρό.
  - Π.χ., HelloWorld
- Χρησιμοποιούμε ολόκληρες λέξεις (και συνδυασμούς τους) για τα ονόματα
  - Δεν πειράζει αν βγαίνουν μεγάλα ονόματα
- Χρησιμοποιούμε το CamelCase Style
  - Όταν για ένα όνομα έχουμε πάνω από μία λέξη, τις συνενώνουμε και στο σημείο συνένωσης κάνουμε το πρώτο γράμμα της λέξης κεφαλαίο
  - printName όχι print_name
- Χρησιμοποιούμε κεφαλαία και '_' για τις σταθερές.
  - Π.χ., PI_NUMBER



Παράδειγμα 2

- Φτιάξτε ένα πρόγραμμα που τυπώνει το λόγο δύο ακεραίων.

Division.java

```
class Division
{
  public static void main(String args[])
  {
    int arithmitis = 32;
    int paronomastis = 10;
    double diairesi;
    diairesi = arithmitis/(double)paronomastis;
    System.out.println("Result = " + diairesi);
  }
}
```

Division.java

```
class Division
{
  public static void main(String args[])
  {
    int arithmitis = 32;
    int paronomastis = 10;
    double diairesi;
    diairesi = arithmitis/(double)paronomastis;
    System.out.println("Result = " + diairesi);
  }
}
```

- Ορισμός μεταβλητών
- Η Java είναι strongly typed γλώσσα: κάθε μεταβλητή θα πρέπει να έχει ένα τύπο
- Οι τύποι int και double είναι πρωταρχικοί (βασικοί) τύποι (primitive types)
- Εκτός από τους βασικούς τύπους, όλοι οι άλλοι τύποι είναι κλάσεις

Πρωταρχικοί τύποι

Όνομα τύπου	Τιμή	Μνήμη
boolean	true/false	1 byte
char	Χαρακτήρας (Unicode)	2 bytes
byte	Ακέραιος	1 byte
short	Ακέραιος	2 bytes
int	Ακέραιος	4 bytes
long	Ακέραιος	8 bytes
float	Πραγματικός	4 bytes
double	Πραγματικός	8 bytes

Όταν ορίζουμε μια μεταβλητή δεσμεύεται ο αντίστοιχος χώρος στη μνήμη.
Το όνομα της μεταβλητής αντιστοιχίζεται με αυτό το χώρο στη μνήμη.

Πρωταρχικοί τύποι

Όνομα τύπου	Τιμή	Μνήμη
boolean	true/false	1 byte
char	Χαρακτήρας (Unicode)	2 bytes
byte	Ακέραιος	1 byte
short	Ακέραιος	2 bytes
int	Ακέραιος	4 bytes
long	Ακέραιος	8 bytes
float	Πραγματικός	4 bytes
double	Πραγματικός	8 bytes

Όταν ορίζουμε μια μεταβλητή δεσμεύεται ο αντίστοιχος χώρος στη μνήμη.
Το όνομα της μεταβλητής αντιστοιχίζεται με αυτό το χώρο στη μνήμη.

Η μνήμη του υπολογιστή

- Η κύρια μνήμη (main memory) του υπολογιστή κρατάει τα δεδομένα (και τις εντολές) για την εκτέλεση των προγραμμάτων
  - Η μνήμη είναι προσωρινή, τα δεδομένα χάνονται όταν ολοκληρωθεί το πρόγραμμα
- Η μνήμη είναι χωρισμένη σε bytes (8 bits)
  - Ο χώρος που χρειάζεται για ένα χαρακτήρα ASCII
- Το κάθε byte έχει μια διεύθυνση, με την οποία μπορούμε να προσπελάσουμε τη συγκεκριμένη θέση μνήμης
  - Random Access Memory (RAM)
  - Σε 32-bit συστήματα μια διεύθυνση είναι 32 bits, σε 64-bit συστήματα μια διεύθυνση είναι 64 bits.

Διεύθυνση μνήμης	Περιεχόμενο μνήμης
0000	'a'
0001	'b'
0010	'c'
0011	'd'
0100	'e'
0101	'f'
0110	'g'
0111	'h'

Αποθήκευση μεταβλητών

- Η κύρια μνήμη (main memory) του υπολογιστή κρατάει τις μεταβλητές ενός προγράμματος
- Μια μεταβλητή μπορεί να απαιτεί χώρο περισσότερο από 1 byte
  - Π.χ., οι μεταβλητές τύπου double χρειάζονται 8 bytes.
  - Η μεταβλητή τότε αποθηκεύεται σε συνεχόμενα bytes στη μνήμη
- Η θέση μνήμης (διεύθυνση) της μεταβλητής θεωρείται το πρώτο byte από το οποίο ξεκινάει η αποθήκευση του της μεταβλητής
  - Στο παράδειγμα μας η μεταβλητή βρίσκεται στη θέση 0000
  - Αν ξέρουμε την αρχή και το μέγεθος της μεταβλητής μπορούμε να τη διαβάσουμε
- Άρα μία θέση μνήμης αποτελείται από μία διεύθυνση και το μέγεθος

Διεύθυνση μνήμης	Περιεχόμενο μνήμης
0000	8.5
0001	
0010	
0011	
0100	
0101	
0110	
0111	

Αποθήκευση μεταβλητών πρωταρχικού τύπου

```
int x = 5;
int y = 3;
```

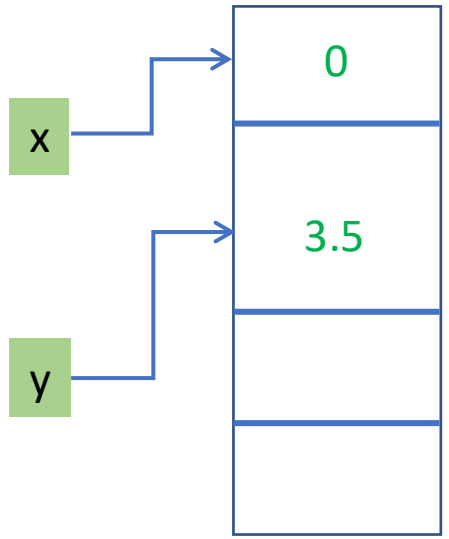
- Για τις μεταβλητές πρωταρχικού τύπου (char, int, double,…) ξέρουμε εκ των προτέρων το μέγεθος της μνήμης που χρειαζόμαστε.
- Όταν ο μεταγλωττιστής δει τη δήλωση μιας μεταβλητής πρωταρχικού τύπου δεσμεύει μια θέση μνήμης αντίστοιχου μεγέθους
  - Η δήλωση μιας μεταβλητής ουσιαστικά δίνει ένα όνομα σε μία θέση μνήμης
  - Συχνά λέμε η θέση μνήμης x για τη μεταβλητή x

	Διεύθυνση μνήμης	Περιεχόμενο μνήμης
x	0000	5
	0001	
	0010	
	0011	
y	0100	3
	0101	
	0110	
	0111	

Αποθήκευση μεταβλητών

- Μπορούμε να σκεφτόμαστε την μνήμη του υπολογιστή σαν μια σειρά από «κουτάκια» διαφόρων μεγεθών στα οποία μπορούμε να αποθηκεύουμε δεδομένα
  - Το κάθε κουτάκι έχει διεύθυνση και περιεχόμενα
- Όταν ορίζουμε μια μεταβλητή πρωταρχικού τύπου (π.χ., `int x`) αυτό που γίνεται είναι ότι:
  - Δεσμεύουμε ένα κουτάκι κατάλληλου μεγέθους
    - 4 bytes για ένα int
  - Δίνουμε στο κουτάκι το όνομα της μεταβλητής
    - Το κουτάκι `x` αντί για το κουτάκι `0110`
  - Γι' αυτό και η μεταβλητή ορίζεται μόνο μια φορά
- Με την ανάθεση αλλάζουμε τα περιεχόμενα του κουτιού
- Αν δεν αρχικοποιήσουμε μια μεταβλητή η Java την αρχικοποιεί στο μηδέν (ή το αντίστοιχο του μηδέν για μη αριθμητικές μεταβλητές)

```
int x;
double y = 3.5;
```



Division.java

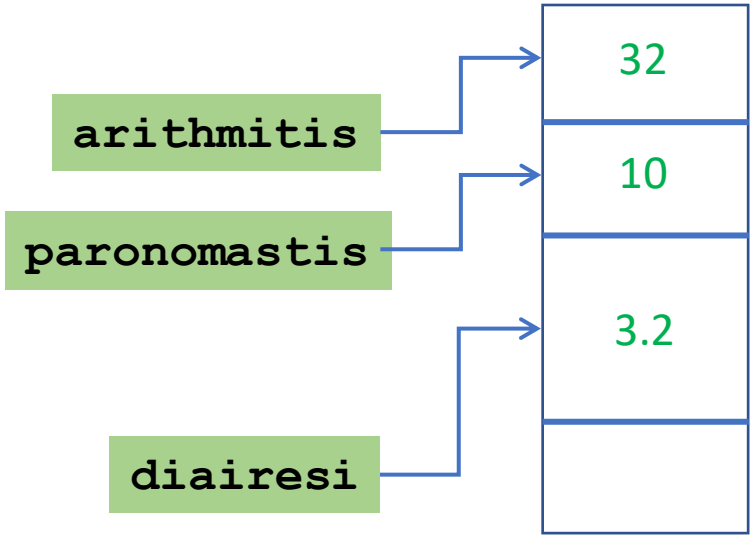
```
class Division
{
  public static void main(String args[])
  {
    int arithmitis = 32;
    int paronomastis = 10;
    double diairesi;
    diairesi = arithmitis/(double)paronomastis;
    System.out.println("Result = " + diairesi);
  }
}
```

Ανάθεση: Η τιμή της έκφρασης στο δεξιό μέλος του "=" ανατίθεται στη μεταβλητή στο αριστερό μέλος

Σε μια ανάθεση το αριστερό μέλος πρέπει να είναι μια μεταβλητή

Division.java

```
class Division
{
  public static void main(String args[])
  {
    int arithmitis = 32;
    int paronomastis = 10;
    double diairesi;
    diairesi = arithmitis/(double)paronomastis;
    System.out.println("Result = " + diairesi);
  }
}
```



Ανάθεση: Διαβάζουμε τα περιεχόμενα των μεταβλητών **arithmitis** και **paronomastis**, κάνουμε τον υπολογισμό και αλλάζουμε τα περιεχόμενα της μεταβλητής **diairesi** αποθηκεύοντας το αποτέλεσμα της διαίρεσης.

Division.java

```
class Division
{
  public static void main(String args[])
  {
    int arithmitis = 32;
    int paronomastis = 10;
    double diairesi;
    diairesi = arithmitis/(double)paronomastis;
    System.out.println("Result = " + diairesi);
  }
}
```

Μετατροπή τύπου (type casting): **(double)paronomastis** μετατρέπει την τιμή της μεταβλητής **paronomastis** σε **double**

Αν **δεν** γίνει η μετατροπή, η διαίρεση μεταξύ ακεραίων μας δίνει ακέραιο

Αναθέσεις

- Στην ανάθεση κατά κανόνα, η τιμή του δεξιού μέρους θα πρέπει να είναι ίδιου τύπου με την μεταβλητή του αριστερού μέρους.
- Υπάρχουν εξαιρέσεις όταν υπάρχει συμβατότητα μεταξύ τύπων

- byte $\rightarrow$ short $\rightarrow$ int $\rightarrow$ long $\rightarrow$ float $\rightarrow$ double
  - Μια τιμή τύπου T μπορούμε να την αναθέσουμε σε μια μεταβλητή τύπου που εμφανίζεται δεξιά του T

- (Σε αντίθεση με την C) ο τύπος boolean δεν είναι συμβατός με τους ακέραιους.

Σταθερές μεταβλητές

- Σταθερές (constant) Μεταβλητές: Μεταβλητές που δεν αλλάζουν κατά τη διάρκεια εκτέλεσης του προγράμματος
- Ορίζονται όπως τις υπόλοιπες μεταβλητές με τη λέξη final σαν πρόθεμα:

```
final datatype CONSTANTNAME = VALUE;
```

- Παραδείγματα

```
final double PI = 3.14159;
final int SIZE = 3;
```

Division.java

```
class Division
{
  public static void main(String args[])
  {
    int arithmitis = 32;
    int paronomastis = 10;
    double diairesi;
    diairesi = arithmitis/(double)paronomastis;
    System.out.println("Result = " + diairesi);
  }
}
```

Ο τελεστής “+” μεταξύ αντικείμενων της κλάσης String συνενώνει (concatenates) τα δύο String
Μεταξύ ενός String και ενός βασικού τύπου, ο βασικός τύπος μετατρέπεται σε String και γίνεται η συνένωση

Strings

- Η κλάση String είναι προκαθορισμένη κλάση της Java που μας επιτρέπει να χειριζόμαστε αλφαριθμητικά.
- Ο τελεστής "+" μας επιτρέπει την συνένωση
- Υπάρχουν πολλές χρήσιμες μέθοδοι της κλάσης String. Η πιο χρήσιμη αυτή τη στιγμή είναι:
  - **equals(String x)**: ελέγχει για ισότητα του αντικειμένου που κάλεσε την μέθοδο και του ορίσματος x
- Άλλες μέθοδοι:
  - length(): μήκος του String
  - trim(): αφαιρεί κενά στην αρχή και το τέλος του string
  - split(char ch): σπάει το string σε πίνακα από strings με βάσει το χαρακτήρα ch
  - Επίσης, μέθοδοι για να βρεθεί ένα υπο-string μέσα σε ένα string, κλπ

Escape sequences

- Για να τυπώσουμε κάποιους ειδικούς χαρακτήρες (π.χ., τον χαρακτήρα ") χρησιμοποιούμε τον χαρακτήρα \ και μετά τον χαρακτήρα που θέλουμε να τυπώσουμε
  - Π.χ., ακολουθία \"
- Αυτό ισχύει γενικά για ειδικούς χαρακτήρες.

\b	Backspace
\t	Tab
\n	New line
\f	Form feed
\r	Return (ENTER)
\”	Double quote
\’	Single quote
\\	Backslash
\ddd	Octal code
\uXXXX	Hex-decimal code

Ρεύματα εισόδου/εξόδου

- Τι είναι ένα ρεύμα? Μια αφαίρεση που αναπαριστά μια πηγή (για την είσοδο), ή ένα προορισμό (για την έξοδο) χαρακτήρων
  - Αυτό μπορεί να είναι ένα αρχείο, το πληκτρολόγιο, η οθόνη
  - Όταν δημιουργούμε το ρεύμα το συνδέουμε με την ανάλογη πηγή, ή προορισμό

Είσοδος & Έξοδος

- Τα βασικά ρεύματα εισόδου/εξόδου είναι έτοιμα **αντικείμενα** τα οποία ορίζονται σαν πεδία (στατικά) της κλάσης **System**
  - System.out
  - System.in
  - System.err
- Μέσω αυτών και άλλων βοηθητικών αντικειμένων γίνεται η είσοδος και έξοδος δεδομένων ενός προγράμματος
- Μια εντολή εισόδου/εξόδου έχει αποτέλεσμα το λειτουργικό να πάρει ή να στείλει χαρακτήρες από/προς την αντίστοιχη πηγή/προορισμό

Έξοδος

- Μπορούμε να καλέσουμε τις μεθόδους του System.out:
  - println(String s): για να τυπώσουμε ένα αλφαριθμητικό s και τον χαρακτήρα '\n' (αλλαγή γραμμής)
  - print(String s): τυπώνει το s αλλά δεν αλλάζει γραμμή
  - printf: Formatted output
    - printf("%d", myInt); // τυπώνει ένα ακέραιο
    - printf("%f", myDouble); // τυπώνει ένα πραγματικό
    - printf("%.2f", myDouble); // τυπώνει ένα πραγματικό με δύο δεκαδικά

Είσοδος

- Χρησιμοποιούμε την κλάση Scanner της Java
  - import java.util.Scanner;

- Αρχικοποιείται με το ρεύμα εισόδου:
  - Scanner in = new Scanner(System.in);

- Μπορούμε να καλέσουμε μεθόδους της Scanner για να διαβάσουμε κάτι από την είσοδο
  - nextLine(): διαβάζει μέχρι να βρει τον χαρακτήρα '\n'
  - next(): διαβάζει το επόμενο String μέχρι να βρει κενό χαρακτήρα
  - nextInt(): διαβάζει τον επόμενο int
  - nextDouble(): διαβάζει τον επόμενο double
  - nextBoolean(): διαβάζει τον επόμενο boolean

Παράδειγμα

Με την εντολή αυτή φέρνουμε την κλάση Scanner μέσα στο πρόγραμμα μας ώστε να μπορούμε να φτιάξουμε αντικείμενα τύπου Scanner

```
import java.util.Scanner;

class TestIO
{
	public static void main(String args[])
	{
		System.out.println("Say Something:");
		Scanner input = new Scanner(System.in);
		String line = input.nextLine();
		System.out.println(line);
	}
}
```

new: δημιουργεί ένα αντικείμενο τύπου **Scanner** με το οποίο μπορούμε πλέον να διαβάζουμε από την είσοδο

- Το αντικείμενο αυτό αναπαριστά το πληκτρολόγιο στο πρόγραμμα μας.
- Ένα αντικείμενο φτάνει για να διαβάσουμε πολλαπλές τιμές.

Παράδειγμα

```
import java.util.Scanner;

class TestIO2
{
   public static void main(String args[])
   {
       Scanner input = new Scanner(System.in);
       double d= input.nextDouble();
       System.out.println("division by 4 = " + d/4);
       System.out.println("1+ (division by 4) = " +1+d/4);
       System.out.printf("1+ (division of %.2f by 4) = %.2f",d, 1+d/4);
   }
}
```

Το + λειτουργεί ως concatenation τελεστής μεταξύ Strings, άρα μετατρέπει τους αριθμούς σε Strings

Τι θα τυπώσει αυτό το πρόγραμμα?

Παράδειγμα

```
import java.util.Scanner;

class TestIO2
{
   public static void main(String args[])
   {
       Scanner input = new Scanner(System.in);
       double d= input.nextDouble();
       System.out.println("division by 4 = " + d/4);
       System.out.println("1+ (division by 4) = " +1+d/4);
       System.out.printf("1+ (division of %.2f by 4) = %.2f",d, 1+d/4);
   }
}
```

```
pilia: ~/Desktop/courses/java/code % java TestIO2
4
division by 4 = 1.0
1+ (division by 4) = 11.0
1+ (division of 4.00 by 4) = 2.00
pilia: ~/Desktop/courses/java/code %
```

Τι θα τυπώσει αυτό το πρόγραμμα?